SOFTWARE TEST PLAN

Student Portfolio Showcase

STP • Project-specific test strategy and test cases

1. Introduction

This plan defines test objectives, scope, approach, environment, schedule, deliverables, responsibilities, risks, suspension/resumption criteria, traceability and reporting for Student Portfolio Showcase.

1.1 Scope

Test the implemented Flask application, portfolio features, publication/access rules, validation, error handling, persistence and CI checks. A feature not implemented or not in agreed scope must be marked not applicable or not run, not passed.

1.2 References

- SRS_Student_Portfolio_Showcase.docx
- SAD_Student_Portfolio_Showcase.docx
- Course Software Engineering Mini-Project Deliverables Part-2 instructions

2. Test items

- Flask app factory and routes
- Authentication if included in final scope
- Profile, skills, projects and achievements modules
- Public portfolio and publication-state handling
- Validation, error handling and SQLite persistence
- pytest suite and GitHub Actions workflow
- README setup/run instructions

3. Features to be tested

Area	SRS mapping	Verification
Startup/health	SPS-AT-001, SPS-FR-020	Smoke test and pytest
Authentication (if implemented)	SPS-FR-001–004; SPS-SEC-001,003	Positive/negative tests
Profile	SPS-FR-005–007	Create/update/retrieve/persist
Portfolio entries	SPS-FR-008–013	CRUD and ownership tests
Publication	SPS-FR-014–016	Public/private access tests
Validation/errors	SPS-FR-017,018; SPS-SEC-004,006	Boundary/malformed/missing-resource tests
Persistence	SPS-FR-019	Restart/reload test
CI	SPS-FR-021,022	PR success and intentional failure
Quality attributes	SPS-NFR-001–008	Performance/usability/compatibility/accessibility/setup review

4. Features not to be tested

- Payment, messaging and native mobile features (out of scope).
- Third-party identity providers or integrations not in implementation.
- Hosting-provider internals beyond team configuration.
- Production-scale load and full penetration testing; only proportionate student-project checks are planned.
- Any feature excluded from final instructor-approved scope.

5. Test approach and strategy

Level/type	Purpose / examples
Unit	Verify individual functions/services such as validators and model/service functions.
Integration	Verify route + service + test database interactions.
System	Verify end-to-end owner and visitor workflows.
Acceptance/UAT	Walk through agreed requirements with evaluator/user.
Functional	Check expected behaviour for valid workflows.
Negative/boundary	Check invalid, missing, oversized and malformed inputs.
Security-oriented	Check authentication, ownership, sessions and information disclosure.
Regression	Re-run existing tests after changes.
Performance	Measure representative local requests and record environment.
Usability/accessibility	Review labels, keyboard use, contrast and clarity.
Compatibility	Check core flows in current Chrome/Firefox where available.
CI/CD	Verify PR triggers lint/tests; deployment only if configured.

5.1 Entry criteria

- Testable build available in shared repository.
- Dependencies and test instructions documented.
- Test environment and synthetic test data available.
- Requirements and expected outcomes agreed.
- Test database isolated from real/personal data.

5.2 Exit criteria

- All in-scope test cases have a recorded status.
- No unresolved critical/high defect without documented acceptance.
- Core acceptance criteria met or deviations documented.
- Lint and automated tests pass on candidate main-branch commit.
- Test summary, failures, limitations and evidence recorded.

6. Test environment and tools

Category	Planned environment
OS	Developer macOS; GitHub Actions ubuntu-latest runner
Runtime	Python 3.11
Application/database	Flask and isolated SQLite test database
Test tools	pytest and Flask test client
Code quality	Ruff
Version control/CI	GitHub, pull requests and GitHub Actions
Browsers	Current Chrome and Firefox where available
Test data	Synthetic users and portfolio records only

7. Test schedule

Milestone	Activity	Proposed owner
T1	Review SRS/SAD and derive cases	Nishanth + team
T2	Prepare environment and synthetic data	Nishanth + backend contributor
T3	Run smoke/unit tests as modules arrive	All developers
T4	Integration, security and regression testing	Nishanth + relevant developers
T5	Verify CI and PR checks	Shashank + QA
T6	Acceptance walkthrough and evidence	Whole team
T7	Test summary and known limitations	Nishanth

8. Test deliverables

- This test plan and test case catalogue.
- Requirement traceability matrix.
- Automated pytest scripts and synthetic test data.
- Test execution log with pass/fail/blocked/not-run status.
- Defect reports with reproduction steps and severity.
- CI workflow evidence, including a successful run and failed-check demonstration.
- Final test summary with coverage, unresolved defects and limitations.

9. Roles and responsibilities

Member	Proposed role	Responsibilities
Nishitha S	Coordinator/integration	Coordinate testable builds, integrate fixes and align requirements/evidence.
Narayan Arun	Architecture/design	Clarify design assumptions and support integration fixes.
Nishanth N	QA/test coordinator	Maintain cases, execute tests, log defects and prepare test summary.
Shashank M P	CI/CD	Maintain workflow, verify PR checks and provide CI evidence.

10. Risks and mitigation

Risk	Impact	Mitigation
Late/incomplete build	Tests cannot complete	Test modules incrementally and smoke-test early.
Changing requirements	Test rework	Keep SRS IDs stable; record approved changes.
Integration conflicts	Regression failures	Branches, PRs, small merges and CI.
Insufficient data	Coverage gaps	Use synthetic fixtures for normal and edge cases.
Time constraints	Unexecuted cases	Prioritise high requirements; document deferred tests honestly.
Environment differences	Inconsistent results	Record Python/dependency versions; use CI as repeatable reference.

11. Assumptions and dependencies

- Team has access to shared GitHub repository and Jira project.
- Python/Flask/SQLite baseline remains unless formally changed.
- Synthetic data and isolated test database can be created.
- Authentication and public deployment remain subject to scope confirmation.
- GitHub Actions and repository permissions are available.

12. Suspension and resumption criteria

12.1 Suspend when

- Environment unavailable or unstable.
- Blocking defect prevents substantial test execution.
- Build/commit under test cannot be identified.
- Test data is corrupted or contains sensitive real information.

12.2 Resume when

- Environment stable and build/commit recorded.
- Blocking defects fixed or workaround documented.
- Test data reset/verified.
- Affected cases rerun and regression checks completed.

13. Test case management and traceability

Record test ID, SRS mapping, build/commit, tester, execution date, actual result, status and defect ID. A test is not passed without evidence.

Test ID	SRS mapping	Test name	Procedure	Expected result	Priority
TC-OPS-01	SPS-FR-020/AT-002	GET /health	Send GET /health.	HTTP 200 and healthy payload.	High
TC-AUTH-01	SPS-FR-001	Registration	Submit valid unique details.	Account created; no plaintext password stored.	High
TC-AUTH-02	SPS-FR-002	Login	Log in using valid credentials.	Authenticated session established.	High
TC-AUTH-03	SPS-FR-003	Invalid login	Submit incorrect credentials.	Access denied with generic error.	High
TC-AUTH-04	SPS-FR-004	Logout	Log out then request protected resource.	Session invalidated; access denied.	High
TC-PROF-01	SPS-FR-005	Create profile	Submit valid profile fields.	Profile stored and retrievable.	High
TC-PROF-02	SPS-FR-006	Edit profile	Update existing owner profile.	Updated values persist.	High
TC-PROF-03	SPS-FR-007	View profile	Open profile view.	Only selected public fields displayed.	High
TC-SKILL-01	SPS-FR-008	Add skill	Add valid skill.	Skill associated with correct portfolio.	Medium
TC-SKILL-02	SPS-FR-009	Edit/remove skill	Edit then remove owned skill.	Changes persist; removed skill disappears.	Medium
TC-PROJ-01	SPS-FR-010	Add project	Add valid project details.	Project stored and displayed.	High
TC-PROJ-02	SPS-FR-011	Edit project	Update owned project.	Updated details persist.	High
TC-PROJ-03	SPS-FR-012	Remove project	Remove owned project.	Project no longer listed.	Medium
TC-ACH-01	SPS-FR-013	Manage achievement	Add/edit/remove achievement.	Each operation behaves as expected.	Medium
TC-PUB-01	SPS-FR-014	Publish/unpublish	Toggle publication state.	State saved correctly.	High
TC-PUB-02	SPS-FR-015	Public view	Visit published portfolio URL.	Content visible without edit controls.	High
TC-PUB-03	SPS-FR-016	Private restriction	Visit unpublished URL as visitor.	Private content not disclosed.	High
TC-VAL-01	SPS-FR-017	Input validation	Submit missing/oversized/malformed values.	Invalid data rejected safely.	High
TC-ERR-01	SPS-FR-018	Error handling	Request missing resource.	Safe error returned; server remains running.	High

TC-DB-01	SPS-FR-019	Persistence	Create record, restart, retrieve.	Record remains available.	High
TC-CI-01	SPS-FR-021	CI success	Open PR with passing lint/tests.	Workflow reports success.	High
TC-CI-02	SPS-FR-022	CI failure	Use temporary failing test on test branch.	Workflow reports failure; do not merge.	High
TC-SEC-01	SPS-SEC-002	Ownership check	User A attempts to modify User B record.	Denied; User B data unchanged.	High
TC-SEC-02	SPS-SEC-005/006	Secrets/error hygiene	Inspect repository and error/log output.	No committed secrets or sensitive stack details.	High